



Dayalbagh Educational Institute

Dayalbagh, Agra

IOT-408 Report File

ScribeNova – A SaaS Platform for AI-Powered Content Generation and Interactive Document Querying

Submitted To:

Mr. Amarjeet Singh Chauhan Sir

Submitted By:

Tarun Kumar: 2303629

Sunny: 2303627

1. Abstract

ScribeNova is an innovative Software as a Service model, which uses artificial intelligence to provide advanced content creation and document interaction options. Distinguished target users are digital-content creators, marketers, and professionals who need fast content creation for sites like YouTube, Instagram, LinkedIn, other digital, and so on. ScribeNova brings a powerful feature “AI Chat with PDF” that helps users upload PDFs and ask questions about them right away. In turn, the AI provides answers comprising of metadata, e.g. the filename, the page and line number, thus improving context.

For the backend logic, Node.js is used; at the frontend, the project uses Next.js; for retrieval augmented generation, Lang Chain, Valkey, Qdrant, and Ollama are used, and Gemini API handles content generation. For secure users’ identification, Clerk is deployed and with the use of Drizzle in conjunction with Neon, the backend database is well attended to. Designing this project, we have the chance to work with and learn about basic principles of full stack web-applications, AI-functionality, conscientiously designed user interface.

Keywords:

2. Introduction

Because people have always needed more customized, flexible content in the online world, users need faster, smarter ways to create content. Also, the processing of massive textual documents such as research papers and reports and technical manuals is not always efficient unless separate querying systems are in place. Through this ScribeNova unifies these issues by way of a generative AI and RAG-based querying for documents.

Objectives:

- To build a scalable SaaS platform that’s meant to create content personalized towards various digital formats.
- To build a feature that will allow users to upload PDF documents and ask those questions to the content of that document in a natural language.
- To maximize usability by adding certain parameters (page, line, PDF name) to add to responses that will assist the user to trawl through their documents.
- To provide a no-less-robust set of security measures with regard to user authentication and session management.
- To bring advanced full-stack technologies in order to ensure scalability and high performance.

3. System Overview with Architecture and Working

Overview:

ScribeNova is an integrated platform that facilitates work with content creators and professionals when dealing with many document-oriented tasks online. It offers two main services:

- Content creation that has been specifically fine-tuned to be used on platforms such as YouTube, LinkedIn and Instagram.
- Using artificial intelligence to analyse content in PDF files to give appropriate responses to user queries.

A responsive platform based on a robust architecture that priorities speed, accuracy, and usability achieves a secured and efficient user experience.

System Architecture:

Dedicated to the design of the ScribeNova, we have a modular foundation comprising three major layers:

Frontend Layer:

- Built with Next.js which is fast in server side rendering (SSR) and dynamic route.
- The interface is implemented using Tailwind CSS for responsive design and develops ShadCN UI components to encourage optimal modular, inclusive experiences.

Backend Layer:

- Engineered with Node.js and Express.js for API endpoint, business logic and PDF conversion.
- Simplified RAG tasks by assigning the production of content and PDF query requests for the backend.

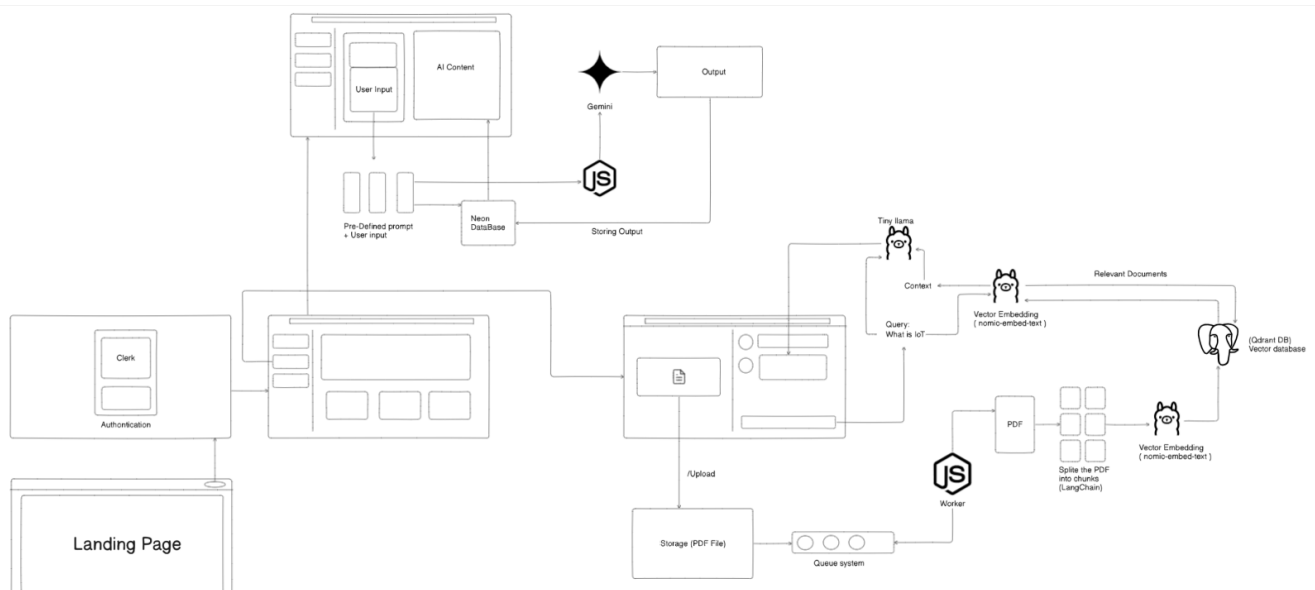
AI and Intelligence Layer:

- Human-like generation of text is implemented using Gemini API with the support of large language models.
- Such tasks as working with prompt templates, PDF extraction, and RAG tasks are performed by LangChain.
- Qdrant makes it possible to perform semantic querying with the help of a vector search on document encodings.
- Ollama which manages the local language model execution improves performance.
- Valkey offers caching and background job support because it handles asynchronous tasks.

Authentication and Database:

- Clerk adds a secured solution to user sign-in and sign-up with RBAC and JWT session tokens to control roles and sessions.
- Drizzle ORM enforces efficient, type-safe database operations and Neon PostgreSQL coordinates database structuring of user, session, upload and query data.

4. Working of the System:



Wire Frame of Working Process

- **User Authentication:** Access to the platform by the users is facilitated by Clerk, a secure logging process in place.
- **Content Generation:** Users provide prompts, which the backend responds by providing the prompts to Gemini API for content creation. The output generated in the contents pop up visually on the user interface.
- **PDF Interaction:** Users upload PDF content, which LangChain manages to break down into manageable chunks. Qdrant is tasked with storing document embeddings which make semantic search possible. Requests to ask questions about the document and to provide the correct answer along with metadata (e.g. page or line number) are generated by the users.
- **Job Queue and Polling:** With Valkey handling these processes, users get instant updates on long tasks via polling without affecting their engagement with complex material.

NOTE:

A robust planning process has been applied to make sure that ScribeNova architecture also accommodates modular, scalable, and maintainable systems. The system comprises loose yet coherent parts – frontend, backend and AI integration – that communicate harmoniously because of well considered API design and service layers. The modular framework speeds up our technological steps to make upgrading and adding abilities easy, if not as simple as changing document types or interfacing with new AI platforms. Moreover, the platform independence is supported by the Docker-based containerization system, which is less cumbersome to implement in diverse environments. This top-level perspective exhibits an innovative design strategy that can be implemented into a working world of SaaS solutions.

5. Benefits of ScribeNova:

- All-in-One Platform:** Combines AI content generation and PDF-based question answering in a single SaaS solution.
- Time-Saving:** Instantly generates platform-specific content and extracts answers from documents, boosting productivity.
- Contextual Responses:** PDF interaction includes metadata like page and line numbers for precise, traceable answers.
- Secure & Scalable:** Built with Clerk authentication, Valkey, and Neon DB for secure and scalable operations.
- Modular Architecture:** Clean separation of concerns ensures easy maintenance, fast deployment, and future expansion.

6. Key Features and Modules

Content Generation Module:

- **Prompt Input:** Users provide specified input such as video titles or keywords to produce outputs.
- **Supported Content Types:**
 - YouTube Descriptions
 - YouTube Tags
 - Instagram Post Ideas
 - LinkedIn Posts
 - Paraphrasing and more.
- **Technology:** The system uses the Gemini API for quick and realistic text outputs.

AI Chat with PDF Module:

- PDF Upload: Documents are submitted by users and analyzed through LangChain and then saved in Qdrant for indexing.
- Querying: Using natural language questions, users are provided with relevant answers, including metadata, like PDF name, page, and line number.

Authentication and User Management:

- Certified by Clerk, it enables account registration, password recovery and streamlined security login.

4. Database and Storage:

- Persistent Storage: All user data, including uploads and queries are stored in Neon PostgreSQL.
- Performance Optimization: With the aid of Valkey to cache metadata, we improve query performance.

7. Challenges and Considerations:

- It is due to the joining of tools such as LangChain, Qdrant and Ollama that complex architectural problems were created.
- Collecting trustable information from the page and line metadata from unevenly formatted PDFs was not easy.
- In an attempt to solve the AI response delays problems, the job queue management with Valkey is implemented.
- Maximizing LLMs' performance in a local environment relied on a proper distribution of the resources.
- The acquisition of access to sessions of the users and records of documents required strong configuration of the Clerk.
- Managing to ensure that response times were kept low and reliable output was delivered was challenging.
- Seeking common ground in UI/UX design between content and PDF modules required considered frontend realization.
- Managing risk of large file uploads destabilizing the server was a significant problem.
- The use of Docker in WSL required enhanced initial setup demands.

8. Conclusion:

ScribeNova illustrates the effective combination of AI and web technologies; to address problems in creating and working with documents. The project provides users with an intuitive way to generate content and search in pdfs, by leveraging semantic queries, through smooth integration of advanced platform such as LangChain, Gemini API, Qdrant, Ollama and Valkey into a strong secured SaaS environment. Due to the modular nature, optimized processes and adaptable responsive interface, the platform guarantees the functionality in the present time and adaptability for time to come. With this project, very important elements of full-stack development, artificial intelligence deployment as well as systems architecture have been thoroughly explored and applied. ScribeNova emphasizes the advantages of intelligent automation in the promotion of more productivity and digital user empowerment within a dynamic information ecosystem.

9. Future Scope:

Support for Additional Document Formats: Provide compatibility with other formats, such as DOCX and TXT, and increase the platform's overall document exchange options.

Multilingual Support: Extend the platform to accommodate the content creation and querying capabilities in several language to cater for global users.

Subscription Plans and Billing Integration: Create a hierarchical subscription model along with embedded payment solutions to render the platform paying to use.

Advanced Analytics Dashboard: Create a strong analytics dashboard for insight into user activity, effectiveness of content and trends of interaction with documents.

Real-Time Collaboration: Have several people interoperate and add in the content creation and document search to promote group productivity.

AI Model Updates: As part of the process, include updated releases, richer training data sets to provide greater accuracy and more options in results.

10. References:

- **Next.js**- A React-based framework that enables server-side rendering, static site generation, and more for fast, dynamic websites.
Documentation: [Next.js Docs](#) GitHub: [Next.js GitHub](#)
Similar Project: [GitHub - Next.js with AI](#) (This repo showcases a Next.js app integrated with AI features.)
- **Node.js**- A JavaScript runtime built on Chrome's V8 engine, used for building scalable network applications.
Documentation: [Node.js Docs](#) GitHub: [Node.js GitHub](#)
Similar Project: [GitHub - Node.js + AI](#) (A sample Node.js app implementing AI features.)
- **Gemini API**- An AI model API by Google, which powers advanced content generation and processing tasks.
Documentation: [Gemini API Docs](#) GitHub: [Gemini API GitHub](#)
Similar Project: [GitHub - AI Content Generator](#) (This repo focuses on building content generators using AI models, similar to your project.)
- **LangChain**- A library designed for building applications with LLMs, enabling advanced document processing and chaining workflows.
Documentation: [LangChain Docs](#) GitHub: [LangChain GitHub](#)
Similar Project: [GitHub - LangChain in Action](#) (Demonstrates using LangChain with document querying and AI models.)
- **Clerk**- A platform for secure user authentication and management, offering seamless login and user session handling.
Documentation: [Clerk Docs](#) GitHub: [Clerk GitHub](#)
Similar Project: [GitHub - Clerk Authentication](#) (An example Next.js app integrated with Clerk for user management.)
- **Qdrant**- A high-performance vector search engine for semantic search and similarity-based document querying.
Documentation: [Qdrant Docs](#) GitHub: [Qdrant GitHub](#)
Similar Project: [GitHub - Qdrant Search System](#) (An example app showing how to use Qdrant for document search and recommendation.)

- **Ollama**- A framework for running and serving large language models (LLMs) locally for efficient inference.
Documentation: [Ollama Docs](#) GitHub: [Ollama GitHub](#)
Similar Project: [GitHub - Ollama for LLMs](#) (A project that demonstrates local LLM inference using Ollama.)
- **Valkey**- A job queue and caching system designed to manage high-throughput tasks, ensuring efficient background processing.
Documentation: [Valkey Docs](#) GitHub: [Valkey GitHub](#)
Similar Project: [GitHub - Job Queue with Node](#) (Example of a task queue and background job processing in a Node.js environment.)
- **Drizzle ORM**- A modern TypeScript ORM for efficient, type-safe database operations, used for managing structured data.
Documentation: [Drizzle ORM Docs](#) GitHub: [Drizzle ORM GitHub](#)
Similar Project: [GitHub - Drizzle ORM Example](#) (Example project using Drizzle ORM with Next.js.)
- **Neon PostgreSQL**- A scalable, serverless PostgreSQL database solution designed for high availability and ease of use.
Documentation: [Neon Docs](#) GitHub: [Neon GitHub](#)
Similar Project: [GitHub - PostgreSQL with Node.js](#) (A project showing PostgreSQL integration with Node.js.)
- **ShadCN UI**- A UI component library built with Tailwind CSS, providing reusable and customizable elements for building modern interfaces.
Documentation: [ShadCN UI Docs](#) GitHub: [ShadCN UI GitHub](#)
Similar Project: [GitHub - Tailwind UI Components](#) (Example of a UI component library using Tailwind CSS.)
- **Express.js**- A web framework for Node.js, simplifying the creation of REST APIs and handling HTTP requests.
Documentation: [Express.js Docs](#) GitHub: [Express.js GitHub](#)
Similar Project: [GitHub - Express API](#) (A simple Express.js-based REST API.)